JobRadar – Project Status Report (12.04.2026)

Общая информация

Проект: JobRadar
Тип: Web-приложение (Backend на Spring Boot)
Цель: система для отслеживания откликов на вакансии

Что сделали:
- **Архитектура** проекта: Controller, Service, Repository, Model
- **Подключение базы данных**: Пока работает, там посмотрим.
Используется **Docker**-контейнер
Подключение через **Spring Data JPA, Hibernate ORM**
- **База данных содержит**:
  - таблицу applications
  - таблицу status_history
  - таблицу users
- **Модель данных** (**Domain Model**): Application, StatusHistory, Role(Enum), User
- **Связи между сущностями**: Application -> User (n:1), StatusHistory -> Application (n:1)
- **Repository**: ApplicationRepository, StatusHistoryRepository, UserRepository extends JpaRepository
- **Service**: ApplicationService: создание заявки, получение всех заявок, получение по id, обновление, удаление, изменение статуса, история изменений
AuthService: регистрация пользователя, проверка уникальности email (exceptions)
- **Controller**: ApplicationController (CRUD, StatusChange), AuthController - POST /api/auth/register
- **Exceptions**: GlobalExceptionHandler (ApplicationNotFoundException, EmailAlreadyExistsException)
- **DTO**: RegisterRequest

Тестирование отдельных узлов:

На текущем этапе было выполнено базовое тестирование backend-логики на уровне controller.

Цель тестирования:
- проверить корректность работы endpoints
- убедиться, что контроллер правильно взаимодействует с service-слоем
- проверить обработку успешных и ошибочных сценариев
- убедиться, что GlobalExceptionHandler корректно возвращает HTTP-ответы

Используемый подход: JUnit, Spring WebMvcTest, MockitoBean, ObjectMapper
Тестирование проводилось как **controller-test**, то есть проверялся web-слой без запуска всей бизнес-логики и без реального обращения к БД

AuthController:

1. Успешная регистрация пользователя

Был реализован тест, который проверяет следующий сценарий:

- клиент отправляет корректный JSON-запрос на регистрацию
- контроллер принимает RegisterRequest
- AuthService возвращает созданного пользователя
- контроллер возвращает HTTP-статус **201 Created**
- в response body возвращаются ожидаемые данные пользователя

Что проверялось в ответе:

- firstname
- lastname
- email
- role

Таким образом было подтверждено, что:

- endpoint регистрации работает корректно
- JSON корректно десериализуется в DTO
- контроллер возвращает правильный HTTP-статус
- успешный сценарий обработки запроса реализован правильно

2. Ошибка при регистрации с уже существующим email

Также был реализован негативный тестовый сценарий:

- клиент отправляет запрос на регистрацию
- AuthService выбрасывает EmailAlreadyExistsException
- GlobalExceptionHandler перехватывает исключение
- клиент получает HTTP-статус **409 Conflict**
- в response body возвращается JSON с сообщением об ошибке

Пример: { "error": "User with Email: ExampleUserName@example.com already exists" }

В результате тестирования было подтверждено, что:

- endpoint регистрации работает в успешном сценарии
- endpoint корректно обрабатывает конфликт email
- GlobalExceptionHandler работает правильно
- web-слой приложения в части регистрации функционирует корректно